	CMOV*
Fetch	icode:ifun = M1[PC]
	rA:rB = M1[PC+1]
	valP = PC +2
Decode	valA = R[rA]
Execute	valE = valA + 0
	Cnd = cond(CC, ifun)
Memory	
Writeback	if (Cnd) R[rB] = valE
PC Update	PC = valP

	IRMOVQ
Fetch	icode:ifun = M1(PC)
	rA:rB = M1(PC + 1)
	valC = M8(PC + 2)
	valP = PC + 10
Decode	
Execute	valE = valC + 0
Memory	
Writeback	R[rB] = valE
PC Update	PC = valP

	MRMOVQ	RMMOVQ
Fetch	icode:ifun = M1(PC)	icode:ifun = M1(PC)
	rA:rB = M1(PC + 1)	rA:rB = M1(PC + 1)
	valC = M8(PC + 2)	valC = M8(PC + 2)
	valP = PC + 10	valP = PC + 10
Decode		valA = R[rA]
	valB = R[rB]	valB = R[rB]
Execute	valE = valB + valC	valE = valB + valC
Memory	valM = M8[valE]	M8[valE] = valA
Writeback	R[rA] = valM	
PC Update	PC = valP	PC = valP

	OPQ
Fetch	icode:ifun = M1(PC)
	rA:rB = M1(PC + 1)
	valP = PC + 2
Decode	valA = R[rA]
	valB = R[rB]
Execute	valE = valB (ifun) valA
	Zflag = (valE == 0)
	Sflag = (valE < 0)
	Oflag = complicated
Memory	
Writeback	R[rB] = valE
PC Update	PC = valP

	PUSHQ	CALL
Fetch	icode:ifun = M1(PC)	icode:ifun = M1(PC)
	rA:rB = M1(PC + 1)	valC = M8(PC + 1)
	valP = PC + 2	valP = PC + 9
Decode	valA = R[rA]	valB = R[%rsp]
	valB = R[%rsp]	
Execute	valE = valB + -8	valE = valB + -8
Memory	M8[valE] = valA	M8[valE] = valP
Writeback	R[%rsp] = valE	R[%rsp] = valE
PC Update	PC = valP	PC = valC

	POPQ	RET
Fetch	icode:ifun = M1(PC)	icode:ifun = M1(PC)
	rA:rB = M1(PC + 1)	valP = PC + 1
	valP = PC + 2	
Decode	valA = R[%rsp]	valA = R[%rsp]
	valB = R[%rsp]	valB = R[%rsp]
Execute	valE = valB + 8	valE = valB + 8
Memory	valM = M8[valA]	valM = M8[valA]
Writeback	R[%rsp] = valE	R[%rsp] = valE
	R[rA] = valM	
PC Update	PC = valP	PC = valM

	J*
Fetch	icode:ifun = M1[PC]
	valC = M8[PC + 1]
	valP = PC + 9
Decode	
Execute	Cnd = cond(CC, ifun)
Memory	
Writeback	
PC Update	PC = Cnd ? valC : valP